async_fifo.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/AsyncFifo/10_geethika/rtl/UnitRtl/AsyncFifo/src) (2 of 7) -

File Edit Tools Syntax Buffers Window Help

async_fifo_pkg.sv | async_fifo.sv | fifo_mem.sv | read_ptr_handler.sv | reset_sync.sv | sync_ff.sv | write_ptr_handler.sv

```
//------------------------------------------------------------------------------
// File: async_fifo.sv
// Description: Top-level asynchronous FIFO with Gray-code pointers and CDC
//              Cummings-style architecture with:
//              - Gray-coded pointers for safe CDC
//              - Two-flop synchronizers (configurable cell type)
//              - True dual-port memory with optional FWFT mode
//              - Programmable almost-full/almost-empty thresholds
//              - Fill count outputs and overflow/underflow detection
//------------------------------------------------------------------------------

module async_fifo #(
  parameter int    DATA_WIDTH       = 32,      // Data bus width
  parameter int    FIFO_DEPTH       = 16,      // FIFO depth (must be power of 2)
  parameter int    SYNC_STAGES      = 2,       // Synchronizer stages (2 or 3)
  parameter int    AF_THRESHOLD     = 14,      // Almost full threshold (DEPTH-2)
  parameter int    AE_THRESHOLD     = 2,       // Almost empty threshold
  parameter bit    FWFT_MODE        = 0,       // First-Word Fall-Through mode
  parameter bit    USE_LIBRARY_SYNC = 0,       // Use ASIC library sync cells
  parameter string SYNC_CELL_NAME   = "SDFFRS" // Library cell name
)(
  // Write Domain Interface
  input  logic                          wclk,          // Write clock
  input  logic                          wrst_n,        // Write reset (active low, async)
  input  logic                          wr_en,         // Write enable
  input  logic [DATA_WIDTH-1:0]         wr_data,       // Write data
  output logic                          wfull,         // Full flag
  output logic                          walmost_full,  // Almost full flag
  output logic [$clog2(FIFO_DEPTH):0]   wfill_count,   // Write domain fill count
  output logic                          overflow_error, // Sticky overflow error

  // Read Domain Interface
  input  logic                          rclk,          // Read clock
  input  logic                          rrst_n,        // Read reset (active low, async)
  input  logic                          rd_en,         // Read enable
  output logic [DATA_WIDTH-1:0]         rd_data,       // Read data
  output logic                          rempty,        // Empty flag
  output logic                          ralmost_empty, // Almost empty flag
  output logic [$clog2(FIFO_DEPTH):0]   rfill_count,   // Read domain fill count
```

```
  output logic                          underflow_error,// Sticky underflow error

  // Error Control (active high, synchronize externally if needed)
  input  logic                          clr_overflow,   // Clear overflow flag (wclk domain)
  input  logic                          clr_underflow   // Clear underflow flag (rclk domain)
);

  //============================================================================
  // Local Parameters
  //============================================================================

  // Calculate address width from FIFO depth
  localparam int ADDR_WIDTH = $clog2(FIFO_DEPTH);
  localparam int PTR_WIDTH  = ADDR_WIDTH + 1;  // Extra bit for wrap detection

  //============================================================================
  // Internal Signals
  //============================================================================

  // Synchronized resets
  logic wrst_n_sync;
  logic rrst_n_sync;

  // Write domain signals
  logic [ADDR_WIDTH-1:0]    waddr;
  logic [PTR_WIDTH-1:0]     wgray;
  logic                     wfull_int;
  logic                     walmost_full_int;
  logic [ADDR_WIDTH:0]      wfill_count_int;
  logic                     overflow_error_int;

  // Read domain signals
  logic [ADDR_WIDTH-1:0]    raddr;
  logic [PTR_WIDTH-1:0]     rgray;
  logic                     rempty_int;
  logic                     ralmost_empty_int;
  logic [ADDR_WIDTH:0]      rfill_count_int;
  logic                     underflow_error_int;
  logic                     rempty_next;  // For FWFT mode
```

```
logic                   underflow_error_int;
logic                   rempty_next;  // For FWFT mode

// Synchronized pointers
logic [PTR_WIDTH-1:0]     wgray_sync;  // Write Gray pointer synchronized to read domain
logic [PTR_WIDTH-1:0]     rgray_sync;  // Read Gray pointer synchronized to write domain

//============================================================================
// Parameter Validation
//============================================================================

initial begin
  // FIFO depth must be power of 2
  assert ((FIFO_DEPTH & (FIFO_DEPTH - 1)) == 0 && FIFO_DEPTH > 0) else
    $fatal(1, "FIFO_DEPTH must be a power of 2, got %0d", FIFO_DEPTH);

  // Minimum depth of 4
  assert (FIFO_DEPTH >= 4) else
    $fatal(1, "FIFO_DEPTH must be at least 4");

  // Sync stages must be at least 2
  assert (SYNC_STAGES >= 2) else
    $fatal(1, "SYNC_STAGES must be at least 2");

  // Threshold validation
  assert (AF_THRESHOLD > 0 && AF_THRESHOLD < FIFO_DEPTH) else
    $fatal(1, "AF_THRESHOLD must be between 1 and FIFO_DEPTH-1");

  assert (AE_THRESHOLD >= 0 && AE_THRESHOLD < FIFO_DEPTH) else
    $fatal(1, "AE_THRESHOLD must be between 0 and FIFO_DEPTH-1");
end

//============================================================================
// Reset Synchronizers
//============================================================================

// Write domain reset synchronizer
reset_sync #(
  .SYNC_STAGES (SYNC_STAGES)
```

```
// Write domain reset synchronizer
reset_sync #(
  .SYNC_STAGES (SYNC_STAGES)
) u_wrst_sync (
  .clk      (wclk),
  .rst_n_in (wrst_n),
  .rst_n_out(wrst_n_sync)
);

// Read domain reset synchronizer
reset_sync #(
  .SYNC_STAGES (SYNC_STAGES)
) u_rrst_sync (
  .clk      (rclk),
  .rst_n_in (rrst_n),
  .rst_n_out(rrst_n_sync)
);

//============================================================================
// Write Pointer Handler
//============================================================================

write_ptr_handler #(
  .ADDR_WIDTH   (ADDR_WIDTH),
  .AF_THRESHOLD (AF_THRESHOLD)
) u_write_ptr (
  .wclk          (wclk),
  .wrst_n        (wrst_n_sync),
  .wr_en         (wr_en),
  .rgray_sync    (rgray_sync),
  .clr_overflow  (clr_overflow),
  .waddr         (waddr),
  .wgray         (wgray),
  .wfull         (wfull_int),
  .walmost_full  (walmost_full_int),
  .wfill_count   (wfill_count_int),
  .overflow_error(overflow_error_int)
);
```

```
//============================================================================
// Read Pointer Handler
//============================================================================

read_ptr_handler #(
  .ADDR_WIDTH   (ADDR_WIDTH),
  .AE_THRESHOLD (AE_THRESHOLD)
) u_read_ptr (
  .rclk           (rclk),
  .rrst_n         (rrst_n_sync),
  .rd_en          (rd_en),
  .wgray_sync     (wgray_sync),
  .clr_underflow  (clr_underflow),
  .raddr          (raddr),
  .rgray          (rgray),
  .rempty         (rempty_int),
  .rempty_next_out(rempty_next),
  .ralmost_empty  (ralmost_empty_int),
  .rfill_count    (rfill_count_int),
  .underflow_error(underflow_error_int)
);

//============================================================================
// Pointer Synchronizers (CDC)
//============================================================================

// Synchronize write Gray pointer to read domain
sync_ff #(
  .WIDTH          (PTR_WIDTH),
  .SYNC_STAGES    (SYNC_STAGES),
  .USE_LIBRARY_SYNC(USE_LIBRARY_SYNC),
  .SYNC_CELL_NAME  (SYNC_CELL_NAME)
) u_wgray_sync (
  .clk   (rclk),
  .rst_n (rrst_n_sync),
  .d     (wgray),
  .q     (wgray_sync)
);
```

```
);

// Synchronize read Gray pointer to write domain
sync_ff #(
  .WIDTH           (PTR_WIDTH),
  .SYNC_STAGES     (SYNC_STAGES),
  .USE_LIBRARY_SYNC(USE_LIBRARY_SYNC),
  .SYNC_CELL_NAME  (SYNC_CELL_NAME)
) u_rgray_sync (
  .clk   (wclk),
  .rst_n (wrst_n_sync),
  .d     (rgray),
  .q     (rgray_sync)
);

//============================================================================
// Dual-Port Memory
//============================================================================

fifo_mem #(
  .DATA_WIDTH (DATA_WIDTH),
  .ADDR_WIDTH (ADDR_WIDTH),
  .FWFT_MODE  (FWFT_MODE),
  .INIT_MEM   (1)  // Enable memory initialization for simulation
) u_fifo_mem (
  // Write port
  .wclk       (wclk),
  .wr_en      (wr_en & ~wfull_int),
  .waddr      (waddr),
  .wdata      (wr_data),
  // Read port
  .rclk       (rclk),
  .rd_en      (rd_en & ~rempty_int),
  .raddr      (raddr),
  .rdata      (rd_data),
  // FWFT control
  .rempty     (rempty_int),
  .rempty_next(rempty_next)
);
```

```
// Output Assignments
//============================================================================

assign wfull          = wfull_int;
assign walmost_full   = walmost_full_int;
assign wfill_count    = wfill_count_int;
assign overflow_error = overflow_error_int;

assign rempty          = rempty_int;
assign ralmost_empty   = ralmost_empty_int;
assign rfill_count     = rfill_count_int;
assign underflow_error = underflow_error_int;

//============================================================================
// Assertions
//============================================================================

// synthesis translate_off

// A4: Gray-code single-bit transition on write pointer
property wgray_single_bit;
  @(posedge wclk) disable iff (!wrst_n_sync)
  !$stable(wgray) |-> $countones(wgray ^ $past(wgray)) == 1;
endproperty
assert property (wgray_single_bit) else
  $error("A4: Write Gray pointer changed more than one bit");

// A4: Gray-code single-bit transition on read pointer
property rgray_single_bit;
  @(posedge rclk) disable iff (!rrst_n_sync)
  !$stable(rgray) |-> $countones(rgray ^ $past(rgray)) == 1;
endproperty
assert property (rgray_single_bit) else
  $error("A4: Read Gray pointer changed more than one bit");

// A2: No write when full
property a2_no_write_when_full;
  @(posedge wclk) disable iff (!wrst_n_sync)
  wfull |-> !wr_en || $stable(waddr);
```

```
  $error("A4: Read Gray pointer changed more than one bit");

// A2: No write when full
property a2_no_write_when_full;
  @(posedge wclk) disable iff (!wrst_n_sync)
  wfull |-> !wr_en || $stable(waddr);
endproperty
assert property (a2_no_write_when_full) else
  $warning("A2: Write enable asserted when FIFO is full");

// A3: No read when empty
property a3_no_read_when_empty;
  @(posedge rclk) disable iff (!rrst_n_sync)
  rempty |-> !rd_en || $stable(raddr);
endproperty
assert property (a3_no_read_when_empty) else
  $warning("A3: Read enable asserted when FIFO is empty");

//------------------------------------------------------------------------
// A5: Full Flag Safety
// The full flag must never fail to assert when FIFO is truly full.
// Uses hierarchical reference to access internal binary pointers.
//------------------------------------------------------------------------
property a5_full_flag_safety;
  @(posedge wclk) disable iff (!wrst_n_sync)
  // When write pointer is exactly DEPTH ahead of read pointer, full must be set
  ((u_write_ptr.wbin - u_write_ptr.rgray_sync_bin) >= FIFO_DEPTH) |-> wfull;
endproperty
assert property (a5_full_flag_safety) else
  $error("A5: Full flag not asserted when FIFO is actually full");

// Cover property: verify we can reach full condition
cover property (@(posedge wclk) disable iff (!wrst_n_sync)
  (u_write_ptr.wbin - u_write_ptr.rgray_sync_bin) == FIFO_DEPTH);

//------------------------------------------------------------------------
// A6: Empty Flag Safety
// The empty flag must never fail to assert when FIFO is truly empty.
//------------------------------------------------------------------------
```

```
// A6: Empty Flag Safety
// The empty flag must never fail to assert when FIFO is truly empty.
//------------------------------------------------------------------------
property a6_empty_flag_safety;
  @(posedge rclk) disable iff (!rrst_n_sync)
  // When Gray pointers are equal, empty must be set
  (wgray_sync == rgray) |-> rempty;
endproperty
assert property (a6_empty_flag_safety) else
  $error("A6: Empty flag not asserted when FIFO is actually empty");

// Cover property: verify we can reach empty condition
cover property (@(posedge rclk) disable iff (!rrst_n_sync)
  wgray_sync == rgray);

//------------------------------------------------------------------------
// Additional Safety Assertions
//------------------------------------------------------------------------

// A7: Overflow error flag assertion
property a7_overflow_detection;
  @(posedge wclk) disable iff (!wrst_n_sync)
  (wr_en && wfull) |=> overflow_error;
endproperty
assert property (a7_overflow_detection) else
  $error("A7: Overflow error not flagged after write when full");

// A8: Underflow error flag assertion
property a8_underflow_detection;
  @(posedge rclk) disable iff (!rrst_n_sync)
  (rd_en && rempty) |=> underflow_error;
endproperty
assert property (a8_underflow_detection) else
  $error("A8: Underflow error not flagged after read when empty");

// A9: Fill count sanity - write domain fill count should never exceed DEPTH
property a9_wfill_count_bounds;
  @(posedge wclk) disable iff (!wrst_n_sync)
  wfill_count <= FIFO_DEPTH;
```

```
    @(posedge wclk) disable iff (!wrst_n_sync)
    (wr_en && wfull) |=> overflow_error;
  endproperty
  assert property (a7_overflow_detection) else
    $error("A7: Overflow error not flagged after write when full");

  // A8: Underflow error flag assertion
  property a8_underflow_detection;
    @(posedge rclk) disable iff (!rrst_n_sync)
    (rd_en && rempty) |=> underflow_error;
  endproperty
  assert property (a8_underflow_detection) else
    $error("A8: Underflow error not flagged after read when empty");

  // A9: Fill count sanity - write domain fill count should never exceed DEPTH
  property a9_wfill_count_bounds;
    @(posedge wclk) disable iff (!wrst_n_sync)
    wfill_count <= FIFO_DEPTH;
  endproperty
  assert property (a9_wfill_count_bounds) else
    $error("A9: Write fill count exceeds FIFO depth");

  // A10: Fill count sanity - read domain fill count should never exceed DEPTH
  property a10_rfill_count_bounds;
    @(posedge rclk) disable iff (!rrst_n_sync)
    rfill_count <= FIFO_DEPTH;
  endproperty
  assert property (a10_rfill_count_bounds) else
    $error("A10: Read fill count exceeds FIFO depth");

  // synthesis translate_on

endmodule : async_fifo
~
~
~
~
~
~
```

```
//------------------------------------------------------------------------------
// File: async_fifo_pkg.sv
// Description: Package containing parameters, types, and functions for
//              the asynchronous FIFO with Gray-code pointers
//------------------------------------------------------------------------------

package async_fifo_pkg;

  //============================================================================
  // Default Parameters
  //============================================================================
  parameter int DEFAULT_DATA_WIDTH  = 32;
  parameter int DEFAULT_FIFO_DEPTH  = 16;  // Must be power of 2
  parameter int DEFAULT_SYNC_STAGES = 2;
  parameter int DEFAULT_AF_OFFSET   = 2;   // Almost full when DEPTH - 2
  parameter int DEFAULT_AE_OFFSET   = 2;   // Almost empty when 2

  //============================================================================
  // Functions
  //============================================================================

  // Calculate the address width from FIFO depth
  function automatic int clog2(int value);
    int result = 0;
    value = value - 1;
    while (value > 0) begin
      result++;
      value = value >> 1;
    end
    return result;
  endfunction

  // Binary to Gray code conversion
  function automatic logic [31:0] bin2gray(logic [31:0] bin);
    return bin ^ (bin >> 1);
  endfunction

  // Gray to Binary code conversion
  function automatic logic [31:0] gray2bin(logic [31:0] gray, int width);
```

```
  // Calculate the address width from FIFO depth
  function automatic int clog2(int value);
    int result = 0;
    value = value - 1;
    while (value > 0) begin
      result++;
      value = value >> 1;
    end
    return result;
  endfunction

  // Binary to Gray code conversion
  function automatic logic [31:0] bin2gray(logic [31:0] bin);
    return bin ^ (bin >> 1);
  endfunction

  // Gray to Binary code conversion
  function automatic logic [31:0] gray2bin(logic [31:0] gray, int width);
    logic [31:0] bin;
    bin = gray;
    for (int i = width - 1; i > 0; i--) begin
      bin[i-1] = bin[i] ^ gray[i-1];
    end
    return bin;
  endfunction

  // Check if value is power of 2
  function automatic bit is_power_of_2(int value);
    return (value > 0) && ((value & (value - 1)) == 0);
  endfunction

endpackage : async_fifo_pkg
~
~
~
~
~
~
~
```

reset_sync.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/AsyncFifo/10_geethika/rtl/UnitRtl/AsyncFifo/src) (5 of 7) - GVIM1 (on ip

File Edit Tools Syntax Buffers Window Help

async_fifo_pkg.sv | async_fifo.sv | fifo_mem.sv | read_ptr_handler.sv | **reset_sync.sv** | sync_ff.sv | write_ptr_handler.sv

```
//------------------------------------------------------------------------------
// File: reset_sync.sv
// Description: Asynchronous reset synchronizer
//              Implements "asynchronous assert, synchronous deassert" pattern
//              Ensures clean reset release aligned to clock edge
//------------------------------------------------------------------------------

module reset_sync #(
  parameter int SYNC_STAGES = 2     // Number of synchronizer stages
)(
  input  logic clk,                 // Domain clock
  input  logic rst_n_in,            // Asynchronous reset input (active low)
  output logic rst_n_out            // Synchronized reset output (active low)
);

  //============================================================================
  // Reset Synchronizer Chain
  //============================================================================

  // Synchronizer stages - synthesis attributes prevent optimization
  (* ASYNC_REG = "TRUE" *)
  logic [SYNC_STAGES-1:0] sync_reg;

  //============================================================================
  // Sequential Logic - Async Assert, Sync Deassert
  //============================================================================

  always_ff @(posedge clk or negedge rst_n_in) begin
    if (!rst_n_in) begin
      // Asynchronous reset assertion - immediate effect
      sync_reg <= '0;
    end else begin
      // Synchronous reset deassertion - shift register pattern
      sync_reg <= {sync_reg[SYNC_STAGES-2:0], 1'b1};
    end
  end

  // Output is the last stage of the synchronizer
  assign rst_n_out = sync_reg[SYNC_STAGES-1];
```

File Edit Tools Syntax Buffers Window Help

async_fifo_pkg.sv async_fifo.sv fifo_mem.sv read_ptr_handler.sv **reset_sync.sv** sync_ff.sv write_ptr_handler.sv

```
  logic [SYNC_STAGES-1:0] sync_reg;

  //============================================================================
  // Sequential Logic - Async Assert, Sync Deassert
  //============================================================================

  always_ff @(posedge clk or negedge rst_n_in) begin
    if (!rst_n_in) begin
      // Asynchronous reset assertion - immediate effect
      sync_reg <= '0;
    end else begin
      // Synchronous reset deassertion - shift register pattern
      sync_reg <= {sync_reg[SYNC_STAGES-2:0], 1'b1};
    end
  end

  // Output is the last stage of the synchronizer
  assign rst_n_out = sync_reg[SYNC_STAGES-1];

  //============================================================================
  // Assertions
  //============================================================================

  // Parameter validation
  initial begin
    assert (SYNC_STAGES >= 2) else
      $fatal(1, "SYNC_STAGES must be at least 2 for proper metastability protection");
  end

endmodule : reset_sync
~
~
~
~
~
~
~
```

```
//------------------------------------------------------------------------------
// File: sync_ff.sv
// Description: Parameterized multi-stage synchronizer for clock domain crossing
//              Used to synchronize Gray-coded pointers across clock domains
//              Supports generic FF or ASIC library cell instantiation
//------------------------------------------------------------------------------

module sync_ff #(
  parameter int    WIDTH           = 1,       // Width of the data bus
  parameter int    SYNC_STAGES     = 2,       // Number of synchronizer stages (2 or 3)
  parameter bit    USE_LIBRARY_SYNC = 0,      // Use ASIC library sync cells
  parameter string SYNC_CELL_NAME  = "SDFFRS" // Library cell name (when USE_LIBRARY_SYNC=1)
)(
  input  logic             clk,     // Destination clock
  input  logic             rst_n,   // Active-low asynchronous reset
  input  logic [WIDTH-1:0] d,       // Input data from source domain
  output logic [WIDTH-1:0] q        // Synchronized output
);

  //============================================================================
  // Synchronizer Implementation Selection
  //============================================================================

  generate
    if (USE_LIBRARY_SYNC) begin : gen_library_sync
      //------------------------------------------------------------------------
      // ASIC Library Cell Instantiation
      // Note: This is a template - actual cell instantiation depends on
      // the target library. Replace the placeholder with actual cells.
      //------------------------------------------------------------------------

      // Intermediate signals for multi-stage sync
      logic [WIDTH-1:0] sync_stage [SYNC_STAGES-1:0];

      // Generate synchronizer chain for each bit
      for (genvar bit_idx = 0; bit_idx < WIDTH; bit_idx++) begin : gen_bit_sync
        for (genvar stage = 0; stage < SYNC_STAGES; stage++) begin : gen_stage

          // Synthesis attributes for metastability-hardened cells
```

File Edit Tools Syntax Buffers Window Help

async_fifo_pkg.sv | async_fifo.sv | fifo_mem.sv | read_ptr_handler.sv | reset_sync.sv | sync_ff.sv | write_ptr_handler.sv

```
    // Generate synchronizer chain for each bit
    for (genvar bit_idx = 0; bit_idx < WIDTH; bit_idx++) begin : gen_bit_sync
      for (genvar stage = 0; stage < SYNC_STAGES; stage++) begin : gen_stage

        // Synthesis attributes for metastability-hardened cells
        (* dont_touch = "true", ASYNC_REG = "TRUE" *)
        logic sync_cell_q;

        if (stage == 0) begin : first_stage
          // First stage samples from source domain
          // Placeholder: Replace with actual library cell instantiation
          // Example: SDFFRS u_sync (.D(d[bit_idx]), .CK(clk), .RN(rst_n), .Q(sync_cell_q));
          always_ff @(posedge clk or negedge rst_n) begin
            if (!rst_n)
              sync_cell_q <= 1'b0;
            else
              sync_cell_q <= d[bit_idx];
          end
        end else begin : subsequent_stage
          // Subsequent stages chain from previous
          always_ff @(posedge clk or negedge rst_n) begin
            if (!rst_n)
              sync_cell_q <= 1'b0;
            else
              sync_cell_q <= sync_stage[stage-1][bit_idx];
          end
        end

        assign sync_stage[stage][bit_idx] = sync_cell_q;

      end : gen_stage
    end : gen_bit_sync

    // Output is the last stage
    assign q = sync_stage[SYNC_STAGES-1];

  end else begin : gen_generic_sync
    //------------------------------------------------------------------------
```

```
  end else begin : gen_generic_sync
    //------------------------------------------------------------------------
    // Generic Flip-Flop Implementation (Default)
    //------------------------------------------------------------------------

    // Synchronizer stages - synthesis attributes prevent optimization
    (* ASYNC_REG = "TRUE" *)
    logic [WIDTH-1:0] sync_reg [SYNC_STAGES-1:0];

    always_ff @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
        for (int i = 0; i < SYNC_STAGES; i++) begin
          sync_reg[i] <= '0;
        end
      end else begin
        // First stage samples the input
        sync_reg[0] <= d;
        // Subsequent stages form the synchronizer chain
        for (int i = 1; i < SYNC_STAGES; i++) begin
          sync_reg[i] <= sync_reg[i-1];
        end
      end
    end

    // Output is the last stage of the synchronizer
    assign q = sync_reg[SYNC_STAGES-1];

  end
endgenerate

//==============================================================================
// Assertions
//==============================================================================

// Parameter validation
initial begin
  assert (SYNC_STAGES >= 2) else
    $fatal(1, "SYNC_STAGES must be at least 2 for proper metastability protection");
```

File Edit Tools Syntax Buffers Window Help

async_fifo_pkg.sv | async_fifo.sv | fifo_mem.sv | read_ptr_handler.sv | reset_sync.sv | sync_ff.sv | **write_ptr_handler.sv**

```
//------------------------------------------------------------------------------
// File: write_ptr_handler.sv
// Description: Write pointer handler for asynchronous FIFO
//              - Manages binary and Gray-coded write pointer
//              - Generates full and almost-full flags
//              - Operates in write clock domain
//              - Provides fill count and overflow detection
//------------------------------------------------------------------------------

module write_ptr_handler #(
  parameter int ADDR_WIDTH   = 4,    // Address width (log2 of FIFO depth)
  parameter int AF_THRESHOLD = 14    // Almost full threshold
)(
  input  logic                  wclk,          // Write clock
  input  logic                  wrst_n,        // Write domain reset (active low)
  input  logic                  wr_en,         // Write enable
  input  logic [ADDR_WIDTH:0]   rgray_sync,    // Synchronized read Gray pointer
  input  logic                  clr_overflow,  // Clear overflow flag

  output logic [ADDR_WIDTH-1:0] waddr,         // Write address for memory
  output logic [ADDR_WIDTH:0]   wgray,         // Gray-coded write pointer (for sync)
  output logic                  wfull,         // Full flag
  output logic                  walmost_full,  // Almost full flag
  output logic [ADDR_WIDTH:0]   wfill_count,   // Approximate fill level (write domain)
  output logic                  overflow_error // Sticky overflow error flag
);

  //============================================================================
  // Local Parameters
  //============================================================================

  localparam int PTR_WIDTH = ADDR_WIDTH + 1;  // Extra bit for wrap detection
  localparam int DEPTH     = 1 << ADDR_WIDTH;

  //============================================================================
  // Internal Signals
  //============================================================================

  logic [PTR_WIDTH-1:0] wbin;           // Binary write pointer
```

```
        for (int i = 0; i < SYNC_STAGES; i++) begin
          sync_reg[i] <= '0;
        end
      end else begin
        // First stage samples the input
        sync_reg[0] <= d;
        // Subsequent stages form the synchronizer chain
        for (int i = 1; i < SYNC_STAGES; i++) begin
          sync_reg[i] <= sync_reg[i-1];
        end
      end
    end

    // Output is the last stage of the synchronizer
    assign q = sync_reg[SYNC_STAGES-1];

  end
endgenerate

//============================================================================
// Assertions
//============================================================================

// Parameter validation
initial begin
  assert (SYNC_STAGES >= 2) else
    $fatal(1, "SYNC_STAGES must be at least 2 for proper metastability protection");
  assert (WIDTH >= 1) else
    $fatal(1, "WIDTH must be at least 1");
end

endmodule : sync_ff
```

write_ptr_handler.sv (/projects/am61_proj_dv/shyd2508dv002/sumedha/ActiveProjects/AsyncFifo/10_geethika/rtl/UnitRtl/AsyncFifo/src

File Edit Tools Syntax Buffers Window Help

async_fifo_pkg.sv | async_fifo.sv | fifo_mem.sv | read_ptr_handler.sv | reset_sync.sv | sync_ff.sv | write_ptr_handler.sv

```
// Internal Signals
//============================================================================

logic [PTR_WIDTH-1:0] wbin;           // Binary write pointer
logic [PTR_WIDTH-1:0] wbin_next;      // Next binary write pointer
logic [PTR_WIDTH-1:0] wgray_next;     // Next Gray write pointer
logic [PTR_WIDTH-1:0] rgray_sync_bin; // Synchronized read pointer in binary
logic [PTR_WIDTH-1:0] fill_level;     // Current fill level

logic                 wfull_next;     // Next full flag value
logic                 walmost_full_next;
logic                 overflow_next;  // Next overflow flag value

//============================================================================
// Binary Pointer Logic
//============================================================================

// Increment write pointer when write is valid (not full)
assign wbin_next = wbin + (wr_en & ~wfull);

// Memory address is lower bits of binary pointer
assign waddr = wbin[ADDR_WIDTH-1:0];

//============================================================================
// Binary to Gray Conversion
//============================================================================

// Convert next binary pointer to Gray code
assign wgray_next = wbin_next ^ (wbin_next >> 1);

//============================================================================
// Gray to Binary Conversion (for fill level calculation)
//============================================================================

// Convert synchronized read Gray pointer to binary
always_comb begin
  rgray_sync_bin[PTR_WIDTH-1] = rgray_sync[PTR_WIDTH-1];
  for (int i = PTR_WIDTH - 2; i >= 0; i--) begin
    rgray_sync_bin[i] = rgray_sync_bin[i+1] ^ rgray_sync[i];
```